
Monash University, Faculty of Information Technology
FIT3080**Applied class : Reinforcement Learning****Exercise 1: Multi-Armed Bandit**

Consider a multi-armed bandit problem with $k = 4$ actions, denoted 1, 2, 3, and 4. Consider applying to this problem a bandit algorithm using ϵ -greedy action selection, sample-average action-value estimates,

$$Q_n(a_k) = \frac{R_1(a_k) + R_2(a_k) + \dots + R_{n-1}(a_k)}{n(a_k) - 1},$$

and initial estimates of $Q_0(a_k) = 0$ for all $\forall k, a_k$ for all a . Note n represents time/iteration index and $n(a_k)$ is the number of times action a_k has been selected. Suppose the initial sequence of actions and rewards is:

at time 1 $a_k = 1$ $R_1(1) = 1$

at time 2 $a_k = 2$ $R_2(2) = 1$

at time 3 $a_k = 2$ $R_3(2) = 2$

at time 4 $a_k = 2$ $R_4(2) = 2$

at time 5 $a_k = 3$ $R_5(3) = 0$

On some of these time steps the ϵ case may have occurred causing an action to be selected at random. On which time steps did this occur? On which time steps could this possibly have occurred?

Exercise 2: Q Learning

Consider a system with two states and two actions. You perform actions and observe the rewards and transitions listed below. Each step lists the current state, reward, action and resulting transition as $S_i; R = r; a_k : S_i \rightarrow S_j$. Perform Q-learning using a learning rate of $\alpha = 0.5$ and a discount factor of $\gamma = 0.5$ for each step. The Q-table entries are initialised to zero.

$$Q(a, s) \leftarrow Q(a, s) + \alpha(R(s) + \gamma \max_{a'} [Q(a', s')] - Q(a, s))$$

S_1 $R = -10$ $a_1: S_1 \rightarrow S_1$

Q	S ₁	S ₂
a ₁		
a ₂		

S₁ R = - 10 a₂: S₁→S₂

Q	S ₁	S ₂
a ₁		
a ₂		

S₂ R = +20 a₂: S₂→S₁

Q	S ₁	S ₂
a ₁		
a ₂		

S₁ R = - 10 a₂: S₁→S₂

Q	S ₁	S ₂
a ₁		
a ₂		

Exercise 3: *SARSA*

Assume we are an agent in a 3x2 grid world, as shown in the below figure. We start at the bottom left node (1) and finish in the top right node (6). When node 6 is reached, we receive a reward of +10 and return to the start for a new episode. On all other actions that will not lead to state 6, the reward is -1.

4	5	finish 6
start 1	2	3

In each state we have four possible actions: up, down, left and right. For each action we move deterministically in the specific direction on the grid. Assume that we cannot take actions that bring us outside the grid.

The current estimates of $Q(s, a)$ are given in the below table:

$Q(1, \text{up})=4$			$Q(1, \text{right})=3$
$Q(2, \text{up})=6$		$Q(2, \text{left})=3$	$Q(2, \text{right})=8$
$Q(3, \text{up})=9$		$Q(3, \text{left})=7$	
	$Q(4, \text{down})=2$		$Q(4, \text{right})=5$
	$Q(5, \text{down})=6$	$Q(5, \text{left})=5$	$Q(5, \text{right})=8$

We will continue updating the Q-table with a SARSA (state-action-reward-state-action) algorithm. Starting from node 2, we have sampled the following trajectory: **2 - up - 5 - right - 6**, after which the trial ended. Update two $Q(s, a)$ entries by filling in the form below (take $\alpha = 0.2$; $\gamma = 0.8$). The SARSA equation is provided:

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$$

Hint: The action-value of a terminal state is 0.

Exercise 4: *Q-Learning for the Maze (Python Code)*

In this exercise, you are required to implement the *Q-Learning* algorithm for the same maze as the last lab. The maze has two terminal states (green and red blocks), but there are small negative rewards, $R = -0.1$, on the non-terminal states. You need to write a Python code that performs the following tasks:

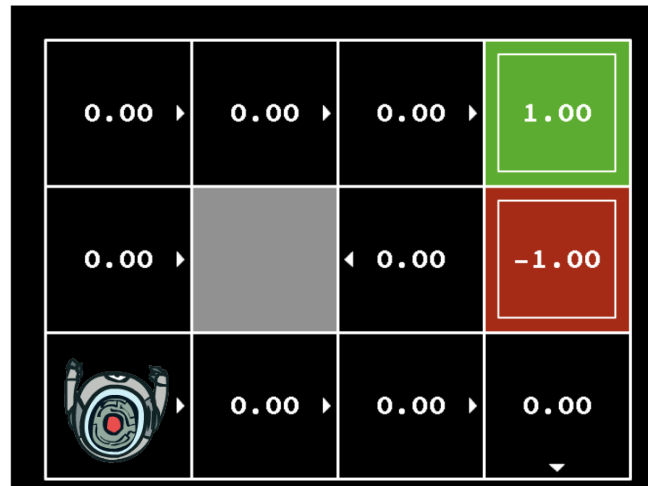


Figure 1: Maze Structure

1. Implement the Q-Learning algorithm with a learning rate α of 0.6 and a discount factor γ of 0.9.
2. Set the start position for the agent at the bottom-most left block of the maze (suggested coordinate: (3, 0)).
3. Initialise the Q-table entries by zero.
4. Calculate the Q-values and policy for each block of the maze.
5. Try different initial blocks of your choice and record your observations.